

CST8509 Lab4 Actor

Overview

We add a travelling red ball actor to the AWS small house Gazebo world simulator in preparation for applying Reinforcement Learning techniques to our real-world Create 3 robot application.

When you have completed this lab, you will know how to

- Add a travelling red ball to the AWS small house world
- Write a ROS 2 node that subscribes to the simulated Create 3 camera
- Detect the red ball in the images from the camera

Instructions

The primary development platform for this lab is your Ubuntu 22.04 loaner laptop. You can possibly also complete this work on other platforms, such as your Ubuntu 22.04 virtual machine, but if you encounter issues you'll need to be prepared to fall back to your loaner laptop.

Adding an actor to the AWS small house world

A tutorial with many details about Classic Gazebo actors can be found here:

<https://classic.gazebosim.org/tutorials?tut=actor>

We will not need to absorb all of the details in order to add an actor. For example, let's add a human actor to the AWS small house world:

- scroll to near the bottom of the page of the above tutorial to find the XML code for one of the actors in the café
- and add that `<actor> ...</actor>` XML element to the AWS small house world.
 - find the file called **small_house.world** in the AWS small house source code
 - add the actor element to the **small_house.world** in an appropriate place (not inside another element)
- rebuild and relaunch, then look for the actor
- after verifying the actor was added, you can comment it out: `<!-- xmlcomment -->`

Add a travelling red ball to the AWS small house world

- Begin by adding the animated box actor from the Example World which is explained in the Scripted Trajectories section of the above tutorial. The actor element begins with `<actor>` and ends with `</actor>`, and it includes a trajectory element.
- Verify the white cube appears in your AWS small house

- Change the box into a red sphere, by swapping this XML in place of the box XML:


```
<visual name="visual">
  <geometry>
    <sphere>
      <radius>.2</radius>
    </sphere>
  </geometry>
  <material name="red">
    <ambient>1 0 0 1</ambient>
    <diffuse>1 0 0 1</diffuse>
    <specular>0 0 0 0</specular>
    <emissive>0 0 0 1</emissive>
  </material>
</visual>
```
- Change the trajectory waypoints so that the ball goes back and forth in a straight line instead of tracing a square

Ros 2 Node to process images

We can implement a ROS 2 node to subscribe to the simulated camera images, detect a red ball, and publish the original image except with the red ball circled. The results of this can be monitored in Rviz by adding another Image with the right ROS 2 topic (which topic? Read the code below).

Add this ROS 2 code to your `aisd_vision` package, as **redball.py**, beside the **hands.py** file, because it does a similar job. Remember to define the **redball** binary in `setup.py`. You can launch this code the same way you would launch the hands module, and it will subscribe to your simulated camera, and publish modified images.

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
import cv2

class RedBall(Node):
    """
    A Node to analyse red balls in images and publish the results
    """
    def __init__(self):
        super().__init__('redball')
        self.subscription = self.create_subscription(
            Image,
            'custom_ns/camera1/image_raw',
            self.listener_callback,
```

```

10)
self.subscription # prevent unused variable warning

# A converter between ROS and OpenCV images
self.br = CvBridge()
self.target_publisher = self.create_publisher(Image, 'target_redball', 10)
self.twist_publisher = self.create_publisher(Twist, 'cmd_vel', 10)

def listener_callback(self, msg):
    frame = self.br.imgmsg_to_cv2(msg)

    # convert image to BGR format (red ball becomes blue)
    hsv_conv_img = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    bright_red_lower_bounds = (110, 100, 100)
    bright_red_upper_bounds = (130, 255, 255)
    bright_red_mask = cv2.inRange(hsv_conv_img, bright_red_lower_bounds,
    bright_red_upper_bounds)

    blurred_mask = cv2.GaussianBlur(bright_red_mask,(9,9),3,3)
# some morphological operations (closing) to remove small blobs
    erode_element = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
    dilate_element = cv2.getStructuringElement(cv2.MORPH_RECT, (8, 8))
    eroded_mask = cv2.erode(blurred_mask,erode_element)
    dilated_mask = cv2.dilate(eroded_mask,dilate_element)

    # on the color-masked, blurred and morphed image I apply the cv2.HoughCircles-method to
    detect circle-shaped objects
    detected_circles = cv2.HoughCircles(dilated_mask, cv2.HOUGH_GRADIENT, 1, 150,
    param1=100, param2=20, minRadius=2, maxRadius=2000)
    the_circle = None
    if detected_circles is not None:
        for circle in detected_circles[0, :]:
            circled_orig = cv2.circle(frame, (int(circle[0]), int(circle[1])), int(circle[2]),
(0,255,0),thickness=3)
            the_circle = (int(circle[0]), int(circle[1]))
            self.target_publisher.publish(self.br.cv2_to_imgmsg(circled_orig))
        else:
            self.get_logger().info('no ball detected')

def main(args=None):

    rclpy.init(args=args)

```

```
redball = RedBall()

rclpy.spin(redball)

# Destroy the node explicitly
# (optional - otherwise it will be done automatically
# when the garbage collector destroys the node object)
redball.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Submission

Submit your **small_house.world** file to Brightspace.

Demonstration

- Show your lab instructor your running simulation with the red ball travelling back and forth.
- Show the modified camera images (after red ball detection) being monitored in Rviz
- Be prepared to answer questions about your code